

第四次上机课

第四次上机课：服务化重构与响应式数据层 —— CinemaFlow 服务中枢

课程主题：Angular Services, RxJS & Reactive State

实验时长：4 小时

实验目标：在第三次上机课已完成的多页面 SPA 基础上，将 CinemaFlow 的数据层全面服务化，使用 RxJS Observable、服务组合与 BehaviorSubject 构建一个可响应、可复用、可扩展的服务中枢。

课前准备

- 完成第三次上机课的全部内容，确保 CinemaFlow 的路由、Dashboard、MovieList、MovieDetail、MovieAdd、About 页面都可以正常导航
- 回顾第 4 讲：服务与依赖注入的核心概念
 - Angular Service 与 @Injectable
 - 依赖注入层级（providedIn: 'root' 与组件级注入）
 - RxJS Observable、of()、subscribe()
 - 常用操作符：map / tap / catchError / finalize
 - BehaviorSubject 与响应式状态管理
 - inject() 函数式依赖注入

本次实验目标

1. **掌握 Service 与 DI 的进阶用法：**使用 providedIn: 'root' 注册服务，并在服务之间相互组合
 2. **理解响应式数据层：**将同步数据接口改写为 Observable，让组件以订阅方式获取数据
 3. **实现服务组合：**构建 MessageService，并在 MovieService 中注入以记录业务日志
 4. **使用 BehaviorSubject 管理状态：**让所有页面订阅同一份电影状态，实现跨页面自动同步
 5. **规范错误处理与加载态：**使用 catchError / finalize 优化用户体验（选做含更深入拓展）
-

实验内容

Phase 1：服务化基础架构搭建（60 分钟）

目标：建立 CinemaFlow 的服务层骨架，将数据接口改造为 Observable。

任务 1.1：规划并创建服务

在第三次上机课已有的目录结构基础上扩展服务层：

```
# 回顾/确认已有的 MovieService
ng g service services/movie --skip-tests=false

# 新建消息服务（用于业务日志与提示）
ng g service services/message --skip-tests=false

# 新建电影状态服务（本次作为响应式状态中心）
ng g service services/movie-state --skip-tests=false
```

目录结构规划：

```
src/app/
├── pages/                                # 来自第三次上机课
│   ├── dashboard/
│   ├── movie-list/
│   ├── movie-detail/
│   ├── movie-add/
│   └── about/
├── components/                          # 复用型组件
│   ├── movie-card/
│   ├── movie-stats/
│   ├── rating-level/
│   └── message-panel/                  # 本次新增：消息面板
├── services/
│   ├── movie.service.ts               # 改造为 Observable 接口
│   ├── message.service.ts            # 新增：业务消息/日志
│   └── movie-state.service.ts         # 新增：BehaviorSubject 状态中心
└── models/
    └── movie.ts
```

任务 1.2：将 MovieService 改造为 Observable 接口

将第二次上机课中同步返回数组的 `getMovies()` 等方法，全部改造为返回 `Observable<...>`：

```

// services/movie.service.ts
import { Injectable, inject } from '@angular/core';
import { Observable, of, throwError } from 'rxjs';
import { delay, map, tap, catchError } from 'rxjs/operators';
import { Movie } from '../models/movie';
import { MessageService } from './message.service';

@Injectable({ providedIn: 'root' })
export class MovieService {
  private readonly messageService = inject(MessageService);

  private movies: Movie[] = [
    { id: 1, title: '星际穿越', director: '诺兰', rating: 9.3, releaseYear: 2014, status: 'showing' },
    { id: 2, title: '盗梦空间', director: '诺兰', rating: 9.4, releaseYear: 2010, status: 'archived' },
    { id: 3, title: '蝙蝠侠：黑暗骑士', director: '诺兰', rating: 9.1, releaseYear: 2008, status: 'showing' }
  ];

  /** 获取全部电影（模拟异步） */
  getMovies(): Observable<Movie[]> {
    return of([...this.movies]).pipe(
      delay(200),
      tap(list => this.messageService.add(`MovieService: 已加载 ${list.length} 部电影`))
    );
  }

  /** 根据 id 获取单部电影 */
  getMovieById(id: number): Observable<Movie | undefined> {
    return of(this.movies.find(m => m.id === id)).pipe(
      delay(150),
      tap(movie => this.messageService.add(
        movie ? `MovieService: 查询到电影 ${movie.title}` : `MovieService: id=${id} 未找到`
      ))
    );
  }

  /** 新增电影 */
  addMovie(movie: Omit<Movie, 'id'>): Observable<Movie> {
    const created: Movie = { ...movie, id: this.nextId() };
    this.movies = [...this.movies, created];
    return of(created).pipe(
      delay(150),

```

```

        tap(m => this.messageService.add(`MovieService: 已新增电影 ${m.title}`))
    );
}

/** 更新电影 */
updateMovie(movie: Movie): Observable<Movie> {
    this.movies = this.movies.map(m => (m.id === movie.id ? movie : m));
    return of(movie).pipe(
        delay(150),
        tap(m => this.messageService.add(`MovieService: 已更新电影 ${m.title}`))
    );
}

/** 删除电影 */
deleteMovie(id: number): Observable<boolean> {
    const target = this.movies.find(m => m.id === id);
    this.movies = this.movies.filter(m => m.id !== id);
    return of(true).pipe(
        delay(150),
        tap(() => this.messageService.add(
            target ? `MovieService: 已删除电影 ${target.title}` : `MovieService:
id=${id} 删除失败`
        ))
    );
}

private nextId(): number {
    return this.movies.length ? Math.max(...this.movies.map(m => m.id)) + 1 :
1;
}
}

```

 要点：所有改动都是**返回值类型**的变化，`delay()` 用于模拟真实网络请求，为下一讲 `HttpClient` 做好铺垫。

任务 1.3：创建 `MessageService` 并接入根组件

`MessageService` 是本次实验引入的第二个服务，它会被注入到 `MovieService` 中，用于演示服务组合：

```

// services/message.service.ts
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class MessageService {

```

```

private _messages: string[] = [];

get messages(): readonly string[] {
  return this._messages;
}

add(message: string): void {
  const timestamp = new Date().toLocaleTimeString();
  this._messages = [`[${timestamp}] ${message}`, ...this._messages].slice(0,
20);
}

clear(): void {
  this._messages = [];
}
}

```

在 `app.component.ts` 模板中（第三次上机课已有的布局之上）新增一个消息面板组件 `<app-message-panel>`，用于实时显示服务产生的日志：

```

// components/message-panel/message-panel.component.ts
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MatIconModule } from '@angular/material/icon';
import { MatButtonModule } from '@angular/material/button';
import { MessageService } from '../../services/message.service';

@Component({
  selector: 'app-message-panel',
  standalone: true,
  imports: [CommonModule, MatIconModule, MatButtonModule],
  template: `
    <aside class="message-panel" *ngIf="messageService.messages.length">
      <header>
        <mat-icon>chat</mat-icon>
        <span>服务消息</span>
        <button mat-icon-button (click)="messageService.clear()">
          <mat-icon>close</mat-icon>
        </button>
      </header>
      <ul>
        <li *ngFor="let msg of messageService.messages">{{ msg }}</li>
      </ul>
    </aside>
  `,
})

```

```

styles: [`
  .message-panel {
    position: fixed;
    right: 16px;
    bottom: 16px;
    width: 320px;
    max-height: 240px;
    overflow-y: auto;
    background: #ffffbe;
    border: 1px solid #f0d070;
    border-radius: 8px;
    padding: 8px 12px;
    font-size: 12px;
    z-index: 2000;
  }
  header { display: flex; align-items: center; gap: 6px; font-weight: 600; }
  header span { flex: 1; }
  ul { list-style: none; padding: 0; margin: 4px 0 0; }
  li { padding: 2px 0; border-bottom: 1px dashed #eee; }
`]
})
export class MessagePanelComponent {
  readonly messageService = inject(MessageService);
}

```

Phase 2：各页面功能改造与服务组合（100 分钟）

目标：将第三次上机课各页面的数据访问统一切换到 Observable 接口，并使用 `async` 管道订阅。

任务 2.1：Dashboard 首页 —— 使用 `async` 管道

让 Dashboard 页面订阅 `MovieService` 返回的 Observable，无需手动 `subscribe` / `unsubscribe`：

```

// pages/dashboard/dashboard.component.ts
import { Component, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink } from '@angular/router';
import { Observable, map } from 'rxjs';
import { MovieService } from '../../services/movie.service';
import { Movie } from '../../models/movie';

```

```

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterLink /* Material imports... */],
  template: `
    <div class="dashboard-container">
      <h1 class="page-title">
        <mat-icon>dashboard</mat-icon>
        欢迎使用 CinemaFlow
      </h1>

      <!-- 使用 async 管道订阅 Observable -->
      <ng-container *ngIf="stats$ | async as stats">
        <div class="stats-row">
          <mat-card><h3>总数</h3><p>{{ stats.total }}</p></mat-card>
          <mat-card><h3>上映中</h3><p>{{ stats.showing }}</p></mat-card>
          <mat-card><h3>平均评分</h3><p>{{ stats.avgRating | number:'1.1-1' }}
</p></mat-card>
        </div>
      </ng-container>

      <mat-card class="recent-movies" *ngIf="recent$ | async as recent">
        <mat-card-header>
          <mat-card-title>最近添加</mat-card-title>
        </mat-card-header>
        <mat-card-content>
          <div *ngFor="let movie of recent" class="movie-item">
            <a [routerLink]="['/movies', movie.id]">{{ movie.title }}</a>
            <span>{{ movie.rating }}分</span>
          </div>
        </mat-card-content>
      </mat-card>
    </div>
  `,
})
export class DashboardComponent {
  private movieService = inject(MovieService);

  readonly stats$ = this.movieService.getMovies().pipe(
    map(list => ({
      total: list.length,
      showing: list.filter(m => m.status === 'showing').length,
      avgRating: list.reduce((sum, m) => sum + m.rating, 0) / (list.length ||
1)
    })))
  );

```

```

    readonly recent$ = this.movieService.getMovies().pipe(
      map(list => [...list].sort((a, b) => b.id - a.id).slice(0, 5))
    );
  }
}

```

🔑 要点：模板中的 `*ngIf="stats$ | async as stats"` 会自动订阅与退订，组件销毁时不会内存泄漏。

任务 2.2: Movie List 电影列表页 —— Observable + 搜索

在第三次上机课搜索功能的基础上，将数据源切换为 Observable 流：

```

// pages/movie-list/movie-list.component.ts
import { Component, OnInit, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ActivatedRoute, Router, RouterLink } from '@angular/router';
import { FormsModule } from '@angular/forms';
import { Observable, combineLatest, map } from 'rxjs';
import { MovieService } from '../../services/movie.service';
import { Movie } from '../../models/movie';

@Component({
  selector: 'app-movie-list',
  standalone: true,
  imports: [CommonModule, RouterLink, FormsModule /* Material */],
  template: `
    <div class="page-container">
      <div class="page-header">
        <h1>电影列表</h1>
        <a mat-raised-button color="primary" routerLink="/add">
          <mat-icon>add</mat-icon>添加电影
        </a>
      </div>

      <mat-form-field appearance="outline" class="search-field">
        <mat-label>搜索电影</mat-label>
        <input matInput [ngModel]="searchTerm"
          (ngModelChange)="onSearchChange($event)"
          placeholder="输入电影名称或导演">
      </mat-form-field>

      <ng-container *ngIf="filtered$ | async as filtered">
        <table mat-table [dataSource]="filtered" class="movie-table">
          <!-- 列定义保持与第三次上机课一致 -->

```

```

        </table>

        <div *ngIf="filtered.length === 0" class="no-results">
            <mat-icon>search_off</mat-icon>
            <p>没有找到匹配的电影</p>
        </div>
    </ng-container>
</div>
,
})
export class MovieListComponent implements OnInit {
    private movieService = inject(MovieService);
    private route = inject(ActivatedRoute);
    private router = inject(Router);

    searchTerm = '';
    filtered$: Observable<Movie[]>;

    ngOnInit(): void {
        this.filtered$ = combineLatest([
            this.movieService.getMovies(),
            this.route.queryParamMap
        ]).pipe(
            map(([movies, params]) => {
                const term = (params.get('search') || '').toLowerCase();
                this.searchTerm = term;
                if (!term) return movies;
                return movies.filter(m =>
                    m.title.toLowerCase().includes(term) ||
                    m.director.toLowerCase().includes(term)
                );
            })
        );
    }

    onSearchChange(value: string): void {
        this.router.navigate(['/movies'], {
            queryParams: value ? { search: value } : {}
        });
    }

    deleteMovie(id: number): void {
        if (!confirm('确定要删除这部电影吗? ')) return;
        this.movieService.deleteMovie(id).subscribe();
    }
}

```

🔑 要点: `combineLatest` 同时响应数据源变化和查询参数变化, 体现响应式数据流的能力。

任务 2.3: Movie Detail 电影详情页 —— 订阅路由参数

结合路由 `paramMap` 与 `MovieService`, 用 `switchMap` 组合两条数据流:

```
// pages/movie-detail/movie-detail.component.ts
import { Component, inject } from '@angular/core';
import { CommonModule, Location } from '@angular/common';
import { ActivatedRoute, Router } from '@angular/router';
import { switchMap, tap, catchError, of } from 'rxjs';
import { MovieService } from '../../services/movie.service';

@Component({
  selector: 'app-movie-detail',
  standalone: true,
  imports: [CommonModule /* Material */],
  template: `
    <div class="detail-container" *ngIf="movie$ | async as movie; else
loading">
      <button mat-stroked-button (click)="location.back()">
        <mat-icon>arrow_back</mat-icon>返回
      </button>

      <mat-card>
        <h1>{{ movie.title }}</h1>
        <p>导演: {{ movie.director }}</p>
        <p>评分: {{ movie.rating }}</p>
        <p>状态: {{ movie.status }}</p>
      </mat-card>
    </div>

    <ng-template #loading>
      <mat-progress-spinner mode="indeterminate"></mat-progress-spinner>
    </ng-template>
  `
})
export class MovieDetailComponent {
  private route = inject(ActivatedRoute);
  private movieService = inject(MovieService);
  readonly location = inject(Location);

  readonly movie$ = this.route.paramMap.pipe(
    switchMap(params => {
      const id = Number(params.get('id'));

```

```

        return this.movieService.getMovieById(id);
    }},
    catchError(() => of(undefined))
);
}

```

任务 2.4: Movie Add 添加电影页 —— 服务调用闭环

将添加电影的提交逻辑改造成订阅 `MovieService.addMovie()` 的形式：

```

// pages/movie-add/movie-add.component.ts (节选)
submit(): void {
    if (!this.form.valid) return;
    this.saving = true;
    this.movieService.addMovie(this.form.getRawValue()).pipe(
        finalize(() => (this.saving = false))
    ).subscribe({
        next: created => {
            this.router.navigate(['/movies', created.id]);
        },
        error: err => {
            this.errorMessage = '添加失败，请稍后重试';
            console.error(err);
        }
    });
}

```

 要点： `finalize()` 保证无论成功或失败都会关闭加载态； `{ next, error }` 形式订阅展示完整错误处理结构。

任务 2.5: About 关于页面 —— 展示服务元数据

让 About 页面从 `MessageService` 读取最近一条日志，作为“服务工作状态”的展示，同时说明本应用使用了哪些服务：

```

// pages/about/about.component.ts (节选)
import { MessageService } from '../services/message.service';

@Component({
    /* ... */
    template: `
        <mat-card>
            <h1>关于 CinemaFlow</h1>
            <p>本应用采用 Angular 服务化架构，注册了以下根级服务： </p>
            <ul>

```

```

    <li><strong>MovieService</strong> -- 提供电影数据的 Observable 接口</li>
    <li><strong>MessageService</strong> -- 记录业务消息日志</li>
    <li><strong>MovieStateService</strong> -- BehaviorSubject 响应式状态中心
  </li>
</ul>

  <p *ngIf="latestMessage">最近一条服务消息: {{ latestMessage }}</p>
</mat-card>
  ,
})
export class AboutComponent {
  private messageService = inject(MessageService);
  get latestMessage(): string | undefined {
    return this.messageService.messages[0];
  }
}

```

Phase 3：响应式状态管理与错误处理（80 分钟）

目标：引入 BehaviorSubject，构建跨页面同步的电影状态中心，并统一错误/加载态处理。

任务 3.1：BehaviorSubject 状态中心

新增 MovieStateService，它持有一个 BehaviorSubject<Movie[]>，所有页面订阅这一份"单一事实来源"：

```

// services/movie-state.service.ts
import { Injectable, inject } from '@angular/core';
import { BehaviorSubject, Observable, map } from 'rxjs';
import { Movie } from '../models/movie';
import { MovieService } from './movie.service';
import { MessageService } from './message.service';

@Injectable({ providedIn: 'root' })
export class MovieStateService {
  private readonly movieService = inject(MovieService);
  private readonly messageService = inject(MessageService);

  private readonly moviesSubject = new BehaviorSubject<Movie[]>([]);
  readonly movies$: Observable<Movie[]> = this.moviesSubject.asObservable();

  readonly count$ = this.movies$.pipe(map(list => list.length));
  readonly showing$ = this.movies$.pipe(

```

```

    map(list => list.filter(m => m.status === 'showing'))
  );

  /** 从 MovieService 加载并注入状态 */
  load(): void {
    this.movieService.getMovies().subscribe(list => {
      this.moviesSubject.next(list);
      this.messageService.add(`MovieState: 已加载 ${list.length} 部电影到状态中心`);
    });
  }

  addLocal(movie: Movie): void {
    const snapshot = this.moviesSubject.value;
    this.moviesSubject.next([...snapshot, movie]);
  }

  removeLocal(id: number): void {
    const snapshot = this.moviesSubject.value;
    this.moviesSubject.next(snapshot.filter(m => m.id !== id));
  }

  updateLocal(movie: Movie): void {
    const snapshot = this.moviesSubject.value;
    this.moviesSubject.next(snapshot.map(m => (m.id === movie.id ? movie : m)));
  }
}

```

在 `app.component.ts` 中启动时调用 `movieStateService.load()`，并将 `Dashboard`、`MovieList` 改为订阅 `movies$`：这样在任意页面新增、删除电影后，所有订阅方都会自动刷新。

 **要点：不可变更新** (`[...snapshot, movie]`) 是响应式状态管理的关键，保证变更检测可靠触发。

任务 3.2：错误处理与加载态

在 `MovieService` 的每个方法后统一追加 `catchError`；在组件层使用 `finalize` 关闭加载态：

```

// services/movie.service.ts (示例补充)
getMovies(): Observable<Movie[]> {
  return of([...this.movies]).pipe(
    delay(200),
    tap(list => this.messageService.add(`MovieService: 已加载 ${list.length} 部电影`)),
    catchError(),
    finalize()
  );
}

```

```

    catchError(err => {
      this.messageService.add(`MovieService: 加载失败 - ${err?.message ??
err}`);
      return of([]); // 降级为空列表
    })
  );
}

```

在组件模板中使用三态展示 Loading / Error / Data：

```

<ng-container *ngIf="movies$ | async as movies; else loadingTpl">
  <ng-container *ngIf="movies.length; else emptyTpl">
    <!-- 正常展示 -->
  </ng-container>
</ng-container>

<ng-template #loadingTpl>
  <mat-progress-spinner mode="indeterminate"></mat-progress-spinner>
</ng-template>

<ng-template #emptyTpl>
  <p>暂无电影数据</p>
</ng-template>

```

任务 3.3：Router 与服务结合（选做）

让 MovieDetailComponent 在进入时顺带更新 MovieStateService 中的"最近浏览"状态，体验路由事件驱动服务更新的典型模式：

```

// 在 movie-detail.component.ts 中：
readonly movie$ = this.route.paramMap.pipe(
  switchMap(params => {
    const id = Number(params.get('id'));
    return this.movieService.getMovieById(id);
  }),
  tap(movie => {
    if (movie) this.movieStateService.markAsVisited(movie.id);
  }),
  catchError(err => {
    this.messageService.add(`详情页加载失败: ${err}`);
    return of(undefined);
  })
);

```

并在 `MovieStateService` 中新增 `markAsVisited(id)` 方法，维护一个 `visitedIds$` `Observable`，供 `Dashboard` 页面展示“最近浏览”列表。

✅ 验收标准

完成以下检查项，确保实验成功：

功能检查

- ☐ `MovieService` 所有方法均返回 `Observable`，原有同步返回已全部替换
- ☐ `Dashboard`、`MovieList`、`MovieDetail` 等页面全部通过 `async` 管道或显式订阅获取数据
- ☐ 新增、删除、更新电影后，`Dashboard` 与 `MovieList` 立即反映最新数据（`BehaviorSubject` 状态同步）
- ☐ `MessageService` 面板能看到来自 `MovieService` 的日志，时间戳正确
- ☐ 右下角消息面板在多次操作后最多保留 20 条记录，不会无限增长
- ☐ `MovieList` 搜索框变化时，`URL queryParams` 与数据过滤结果同步更新
- ☐ `MovieDetail` 页面切换 `id` 时数据自动刷新（基于 `paramMap + switchMap`）
- ☐ 404 / 路由兜底仍与第三次上机课保持一致

代码质量检查

- ☐ 所有服务使用 `@Injectable({ providedIn: 'root' })` 注册
- ☐ 服务之间通过构造器或 `inject()` 注入，无循环依赖
- ☐ 组件不再持有可变的本地数组状态，数据源统一来自服务的 `Observable`
- ☐ 至少使用了 `map` / `tap` / `switchMap` / `catchError` / `finalize` 中的 4 个操作符
- ☐ 至少一处使用 `BehaviorSubject` 暴露 `xxx$` 形式的公共 `Observable`
- ☐ 所有 `*ngFor` / `*ngIf` 中的 `Observable` 使用 `| async` 管道消费

体验优化

- ☐ 加载阶段显示 `mat-progress-spinner` 或占位内容，不会出现空白闪烁
- ☐ 出错时显示友好提示，不会让应用卡死
- ☐ 服务消息面板可关闭、可自动滚动
- ☐ 跨页面操作后数据保持一致（通过状态中心）

💡 提示与参考

常用 RxJS 操作速查

操作	代码示例
创建常量流	<code>of([...]) / of(value)</code>
模板中订阅	<code>*ngIf="data\$ \ async as data"</code>
手动订阅	<code>this.obs\$.subscribe({ next, error, complete })</code>
映射数据	<code>.pipe(map(list => list.length))</code>
基于上游切换新的流	<code>.pipe(switchMap(id => this.service.get(id)))</code>
副作用（日志/状态）	<code>.pipe(tap(x => console.log(x)))</code>
捕获错误并降级	<code>.pipe(catchError(err => of(defaultValue)))</code>
无论成败都执行	<code>.pipe(finally(() => this.loading = false))</code>
发布新状态	<code>this.subject.next(newValue)</code>
读取当前状态（同步）	<code>this.subject.value</code>

常见问题

1. **视图不刷新**：检查是否遗漏了 `| async` 管道，或者 `BehaviorSubject` 是否调用了 `.next(...)`
2. **Observable 不发出值**：没有订阅就不会执行，`.subscribe()` 或 `async` 管道必不可少
3. **多次订阅导致多次请求**：多次订阅同一个源 `Observable` 会多次触发，必要时使用 `shareReplay(1)`
4. **不可变更新被忽略**：一定要用 `[...arr]`、`{...obj}` 返回新引用，不要原地 `push`
5. **内存泄漏**：手动 `subscribe` 的组件务必在 `ngOnDestroy` 中 `unsubscribe`，优先使用 `async` 管道

拓展挑战（选做）

1. **HttpClient 预演**：将 `of(this.movies)` 替换为 `HttpClient.get('assets/movies.json')`，把 mock 数据放到 `src/assets/movies.json`，体验真实异步
2. **loading/error 信号状态**：在 `MovieStateService` 中维护 `loading$` 和 `error$` 两个 `BehaviorSubject`，供全局使用
3. **Service Guard**：编写一个路由 `CanActivate` 守卫，在进入 `/movies/:id` 前先通过 `MovieService.getMovieById(id)` 检查是否存在，不存在则重定向到列表页

4. **Interceptor 雏形**: 自行实现一个简单的"请求计数器"服务, 每次 `MovieService` 方法被调用都 `+1`, 在 About 页显示累计调用次数